MAFIA
REVENGE
Space to Start

Contents

- Targeting System
- Interaction System
- Focus Tags
- Physics System
- Health and Damage

Targeting System

- Tracks distance from starting entity to target entity
- Uses Camera component to accurately track mouse
- Targeting system sets up Target components' DeltaX and DeltaY variables
- They are then used by rotation system to rotate sprites to face the target
- Also used to calculate unit vectors by putting these variables in a vector and normalizing it
- Helpful for defining projectile trajectory

```
struct Target {
    Entity* target = nullptr;
    SDL_FPoint targetCenter{};
    SDL_FPoint startingCenter{};
    float deltaX{};
    float deltaY{};
};
```

```
Vector2D normalizedDir = {playerTarget.deltaX, playerTarget.deltaY};
normalizedDir.normalize();
bullet.addComponent<Velocity>(Vector2D(normalizedDir.x, normalizedDir.y), 600.0f);
```

Using target component to calculate projectile trajectory

```
float angleRadians = atan2(target.deltaY, target.deltaX);
float angleDegrees = angleRadians * 180.0f / M_PI;

transform.rotation = angleDegrees;
```

Using target component to calculate angle of rotation

Interaction system

- Determines when an entity can interact with another entity
- Mainly used to determine if player can get in the car or not
- Works alongside Action Response System to make action events happen

```
if (e.state == CollisionState::Enter) {
    if (player->hasComponent<Interactable>() && other->hasComponent<Interactable>()) {
        player->getComponent<Interactable>().interactable = true;
        other->getComponent<Interactable>().interactable = true;
    }
}
else if (e.state == CollisionState::Exit) {
    if (player->hasComponent<Interactable>() && other->hasComponent<Interactable>()) {
        player->getComponent<Interactable>().interactable = false;
        other->getComponent<Interactable>().interactable = false;
        other->getComponent<Acceleration>().isAccelerating = false;
    }
}
```

Checking if player is close enough to interact with the car

```
if (e->hasComponent<Interactable>()) {
    auto& i :Interactable& = e->getComponent<Interactable>();
    if (event.type == SDL_EVENT_KEY_DOWN && event.key.key == SDLK_E && i.interactable)
        world.getEventManager().emit(PlayerActionEvent{e.get(), PlayerAction::CarInteract});
}
```

Emitting an action when the player interacts with the car

```
if (e.action == PlayerAction::CarInteract) {
    //things to do to player
    if (e.player->hasComponent<PlayerTag>()) {
        auto& player :Entity* const& = e.player;
        auto& playerSprite = player->getComponent<Sprite>();
        playerSprite.visible = !playerSprite.visible;
        auto& playerKeyboardTag = player->getComponent<KeyboardFocusTag>();
        playerKeyboardTag.active = !playerKeyboardTag.active;
        auto& playerCameraTag = player->getComponent<CameraFocusTag>();
        playerCameraTag.active = !playerCameraTag.active;
        auto& vel :Velocity& = player->getComponent<Velocity>();
        vel.direction.x = vel.direction.y = 0;
    }

    //things to do for car
    else if (e.player->hasComponent<CarTag>()) {
        auto& car :Entity* const& = e.player;
        auto& carKeyboardTag = car->getComponent<KeyboardFocusTag>();
        carKeyboardTag.active = !carKeyboardTag.active;
        auto& carCameraTag = car->getComponent<CameraFocusTag>();
        carCameraTag.active = !carCameraTag.active;
    }
}
```

Defining car interaction logic

Focus Tags

CameraFocusTag: Ensures that the camera is focused on the correct entity

KeyboardFocusTags: Ensures that the correct controls are being used depending on if the player or car is being controlled

```
player.addComponent<CameraFocusTag>(true);
player.addComponent<KeyboardFocusTag>(true);
```

```
car.addComponent<CameraFocusTag>();
car.addComponent<KeyboardFocusTag>();
```

Adding focus tags to player and car

```
//Car movement controls
else if (e->hasComponent<CarTag>() && e->hasComponent<KeyboardFocusTag>() && e->getComponent<KeyboardFocusTag>().active) {
```

```
//find the player
for (auto& e :const unique_ptr<Entity>& : entities) {
    if (e->hasComponent<CameraFocusTag>() && e->getComponent<CameraFocusTag>().active) {
        playerEntity = e.get();
        break;
    }
}
```

Using focus tags to ensure proper controls and camera view are being set

Physics System

- Allows entities to gradually speed up, slow down and brake
- Gradually increases and decreases speed based on acceleration and brake factors
- Runs every update, so speed slowly gets bigger or smaller incrementally

```
//direction and speed
struct Velocity {
    Vector2D direction{};
    float speed{};
    float maxSpeed{};
};

struct Acceleration {
    float accelerationFactor = 1.0f;
    //controls which direction the object is facing
    //allows object to move south while facing north as if it is reversing
    Direction direction{};
    bool isAccelerating{};
};

struct Brake {
    float brakeForce = 1.0f;
    bool isBraking{};
};
```

```
enum Direction {
    NORTH, EAST, SOUTH, WEST, STOP
};

static constexpr float ZERO_EPSILON = 0.4f;
static constexpr float JUMP_START = 0.5f;
```

```
//decelerating
if (!a.isAccelerating) {
    if (v.speed < ZERO_EPSILON && v.speed > -ZERO_EPSILON) { //if speed is 0
        v.speed = 0.0f;
    }
    else {
        float accelFactor = b.isBraking ? a.accelerationFactor * b.brakeForce : a.accelerationFactor;
        accelFactor = v.speed < 0.0f ? -accelFactor : accelFactor;
        v.speed -= accelFactor * dt;
    }
}
//accelerating
else if (a.isAccelerating) {
    //change to account for +ve speeds
    v.speed += (v.speed <0.0f ? -a.accelerationFactor : a.accelerationFactor) * dt;

    if (v.speed >= v.maxSpeed) {
        v.speed = v.maxSpeed;
    }
    if (v.speed <= -v.maxSpeed) {
        v.speed = -v.maxSpeed;
    }
}
```

Defining car interaction logic

Health and Damage

```
struct Health {
    int currentHealth{};
    int maxHealth{};
};

struct Damage {
    int damage{};
};
```

```
else if (tag == "enemy_projectile") {
    if (e.state != CollisionState::Enter) return;

    if (player->hasComponent<Health>() && other->hasComponent<Damage>()) {
        auto& playerHealth = player->getComponent<Health>();
        auto& damage = other->getComponent<Damage>();

        playerHealth.currentHealth -= damage.damage;

        if (playerHealth.currentHealth <= 0) {
            Game::onSceneChangeRequest( "level1");
        }
    }
}
```

Collider logic for when enemy bullet hits player